PROJECT REPORT: TRAFFIC CONGESTION DETECTOR

Submitted By: Name: Mrigank Pandey

Registration Number: 25BCE10158

Branch: B.Tech - Computer Science and Engineering

Year: First Year (2025-26)

Institution: VIT Bhopal University

---

1. Abstract

Urbanization has led to a significant increase in vehicle density, resulting in severe traffic congestion, increased carbon emissions, and economic loss. This project, the **Traffic Congestion Detector**, proposes an automated system to monitor and categorize traffic flow. By utilizing [mention technology, e.g., Ultrasonic sensors / OpenCV], the system provides real-time data on road occupancy, allowing for smarter traffic management and reduced wait times at intersections.

2. Introduction

The objective of this project is to create a functional prototype that can distinguish between "Low," "Medium," and "High" traffic density. Unlike traditional timer-based signals, this system offers a dynamic approach to traffic control.

- **Problem Statement:** Conventional traffic lights operate on fixed timers, leading to "ghost waiting" where drivers wait at red lights despite no cross-traffic.
- **Proposed Solution:** A real-time detection system that triggers alerts or adjusts signal timings based on the actual number of vehicles present.

3. System Architecture

The system is divided into three primary layers:

1. **Sensing Layer:** [e.g., IR Sensors, Ultrasonic Sensors, or a Pi Camera] to detect the presence of vehicles.
2. **Processing Layer:** An Arduino, Raspberry Pi, or Laptop running Python to calculate the percentage of road occupancy.

3. **Output Layer:** A display or LED interface that indicates the current congestion status.

4. Hardware/Software Requirements

4.1 Hardware

- **Microcontroller:** [e.g., Arduino Uno / ESP32 / Raspberry Pi]
- **Sensors:** [e.g., 3x Ultrasonic Sensors or 1x USB Camera]
- **Indicators:** Red, Yellow, and Green LEDs
- **Power Source:** 5V-12V DC Supply

4.2 Software

- **Programming Language:** [e.g., C++ for Arduino / Python]
- **Libraries:** [e.g., OpenCV, NumPy, or LiquidCrystal]
- **Development Environment:** [e.g., Arduino IDE / VS Code]

5. Methodology

1. **Calibration:** The sensors are placed at specific intervals along a model road.
2. **Data Acquisition:** Sensors continuously measure distance. A decrease in distance (or a change in pixel intensity for cameras) signifies a vehicle's presence.
3. **Thresholding:** * **Level 1 (Clear):** No sensors triggered / < 20% occupancy.
    - **Level 2 (Moderate):** 50% of sensors triggered / 50% occupancy.
    - **Level 3 (Heavy):** All sensors triggered / > 80% occupancy.
4. **Action:** The system updates the output display and can be integrated with a signal-switching algorithm.

6. Implementation & Code

(You should paste your specific code here. Below is a placeholder structure)

Python
```
# Sample Logic for Congestion Detection
def detect_congestion(vehicle_count):
    if vehicle_count >= 10:
        return "HEAVY CONGESTION"
    elif vehicle_count >= 5:
        return "MODERATE TRAFFIC"
    else:
        return "CLEAR"
```

7. Results and Discussion

During testing, the prototype successfully identified different traffic scenarios with [mention accuracy, e.g., 90%] accuracy. The system proved to be responsive to sudden changes in traffic flow. Future iterations could include a cloud-based dashboard to share real-time data with city commuters via a mobile app.

8. Conclusion

The Traffic Congestion Detector serves as a foundation for Smart City infrastructure. By transitioning from static to data-driven traffic management, cities can improve fuel efficiency and significantly reduce travel time for citizens.

9. References

1. *Smart Traffic Management Systems*, Journal of Urban Engineering.
2. *Object Detection for Autonomous Systems*, [Author Name/Year].
3. Technical documentation for [mention your sensor, e.g., HC-SR04].

10. Python Code

Traffic Congestion Predictor.py > ...

```python
import pandas as pd
from sklearn.tree import DecisionTreeClassifier
Click to collapse the range.
print("Starting my traffic project...")

data = {
    "hour": [8, 9, 17, 18, 22, 6, 14, 16, 20, 7, 12, 19, 10, 15],
    "day": ["weekday", "weekday", "weekday", "weekday", "weekend", "weekday", "weekend", "weekday", "weekend", "weekday", "weekday", "weekday", "weekday", "weekend"],
    "weather": ["clear", "clear", "clear", "rainy", "clear", "foggy", "clear", "rainy", "clear", "foggy", "clear", "rainy", "clear", "clear"],
    "road": ["city", "city", "highway", "city", "highway", "city", "highway", "city", "highway", "city", "city", "highway", "city", "highway"],
    "traffic": ["high", "high", "high", "high", "low", "low", "medium", "high", "medium", "low", "medium", "high", "medium", "medium"]
}

df = pd.DataFrame(data)

df["day"] = df["day"].map({"weekday": 0, "weekend": 1})
df["weather"] = df["weather"].map({"clear": 0, "rainy": 1, "foggy": 2})
df["road"] = df["road"].map({"city": 0, "highway": 1})
df["traffic"] = df["traffic"].map({"low": 0, "medium": 1, "high": 2})

x_train = df[["hour", "day", "weather", "road"]]
y_train = df["traffic"]

my_model = DecisionTreeClassifier()
my_model.fit(x_train, y_train)

print("model is trained now")

while True:
    print("--- Traffic Check ---")

    h_val = input("enter hour 0-23: ")
    try:
        h = int(h_val)
    except:
        h = 12

    d_val = input("day (weekday/weekend): ")
    if d_val == "weekday":
        d = 0
    else:
        d = 1

    w_val = input("weather (clear/rainy/foggy): ")
```

```python
        d = 1

    w_val = input("weather (clear/rainy/foggy): ")
    if w_val == "clear":
        w = 0
    elif w_val == "rainy":
        w = 1
    else:
        w = 2

    r_val = input("road (city/highway): ")
    if r_val == "highway":
        r = 1
    else:
        r = 0

    p = my_model.predict([[h, d, w, r]])

    print("result:")
    if p[0] == 0:
        print("traffic is low")
        print("tip: its a good time to go")
    if p[0] == 1:
        print("traffic is medium")
        print("tip: leave early")
    if p[0] == 2:
        print("traffic is high")
        print("tip: wait a bit")

    ans = input("again? (y/n): ")
    if ans == "n":
        break

print("program ended")
```